Spark vs. Ray: The Data Engineering Duel

DA3408 Assignment 3

1 Introduction

In this assignment I cleaned the NYC Yellow Taxi trip data with the same pipeline in two frameworks, Apache Spark (PySpark) and Ray Data. Both ran on a two-worker cluster built with Docker on one VirtualBox VM. I measured total time, CPU and memory use, and the cost of a custom Python function (UDF). The two outputs were also checked against each other row by row.

2 Cluster Setup

Both clusters were set up by hand with Docker Compose: one master (or head) container and two worker containers. Figure 1 shows the Spark Master UI with 2 alive workers and the Ray Dashboard with 3 alive nodes (1 head and 2 workers).

	Spark 3.5.3	**Ray (Ray Data)**
Nodes	1 master + 2 workers	1 head + 2 workers
CPU	2 cores per worker (4 total)	2 CPUs per worker (4 total); head runs no tasks
Memory	4 GiB per worker, 3 GiB per executor	4 GiB per worker, 3 GiB head, 1.4 GB object store per worker
Input	6 Parquet files (taxi data)	same files

Table 1: Cluster configuration.

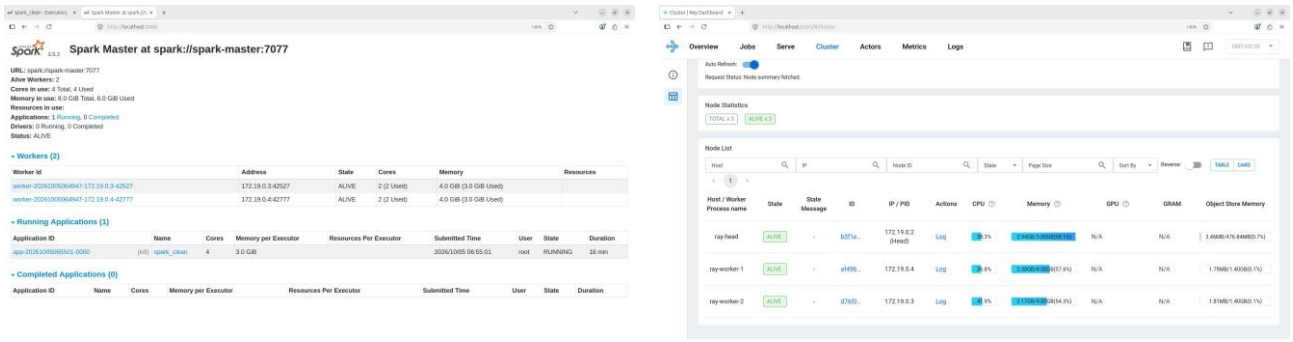


Figure 1: Left: Spark Master UI (2 alive workers, 4 cores). Right: Ray Dashboard cluster view (3 nodes alive).

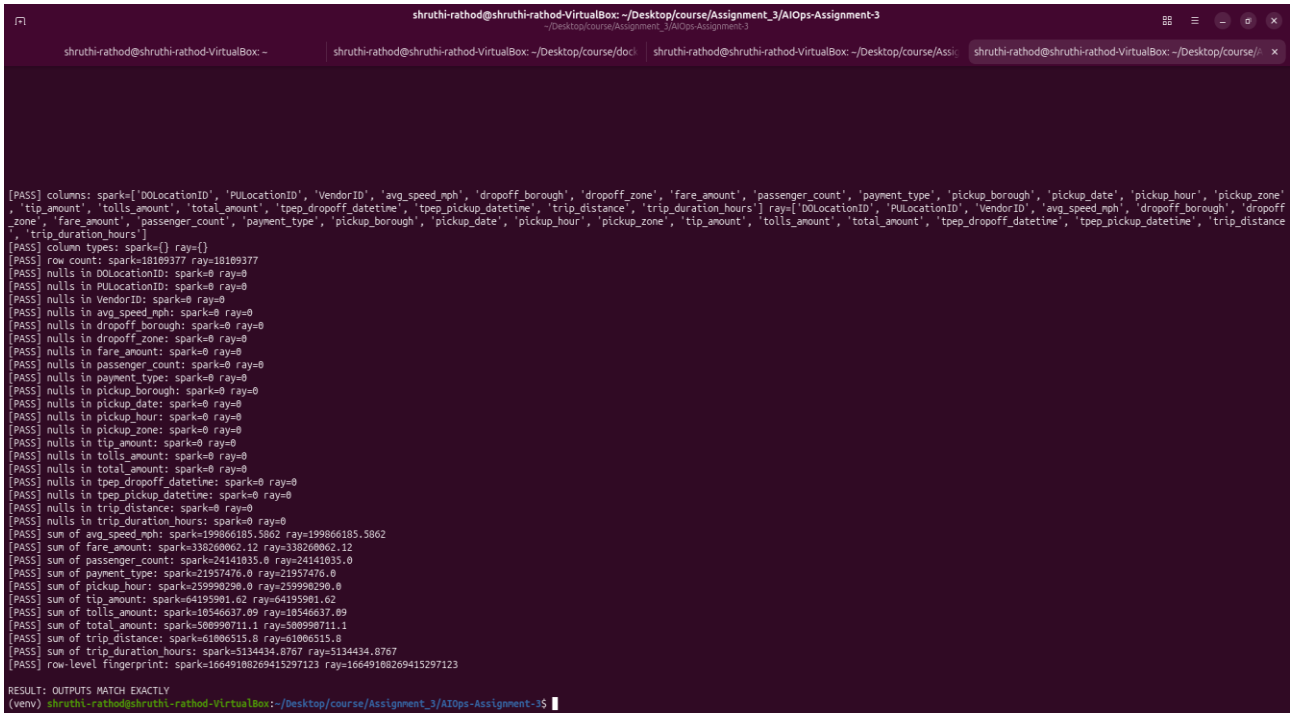
3 The Pipeline and Parity

Both scripts (spark clean.py and ray clean.py) apply the same rules in the same order: (1) cast all columns to one fixed schema, (2) drop rows with nulls in key columns, (3) keep only trips with pickup date in 2022–2024, (4) keep trips with duration > 0 and ≤ 24 h and distance > 0 and ≤ 100 miles, (5) remove duplicate rows, (6) add pickup date and hour, (7) join with the taxi zone lookup table for pickup and drop-off borough and zone, (8) compute average speed with a Python function (UDF) and drop speeds above 100 mph, and (9) write Parquet.

I wrote compare results.py to check that the outputs are really the same. It compares column names, types, row counts, null counts, column sums, and an order-independent hash of every row. Table 2 shows that everything matches, so the outputs are identical.

Check	**Spark**	**Ray**	**Result**
Row count	18,109,377	18,109,377	PASS
Columns and types	20 columns	20 columns	PASS
Null counts (all columns)	0	0	PASS
Sum of total amount	500,990,711.10	500,990,711.10	PASS
Sum of avg speed mph	199,866,185.5862	199,866,185.5862	PASS
Row-level fingerprint	16649108269415297123	16649108269415297123	PASS

Table 2: Parity check output (OUTPUTS MATCH EXACTLY).



4 Performance Results

4.1 Execution time

Total time is the wall-clock time from reading the files to finishing the Parquet write. Spark was about 2.7 times faster.

Framework	Rows out	Total time (s)	Total time (min)
Spark	18,109,377	492.09	8.2
Ray	18,109,377	1340.45	22.3

Table 3: End-to-end time, one run each.

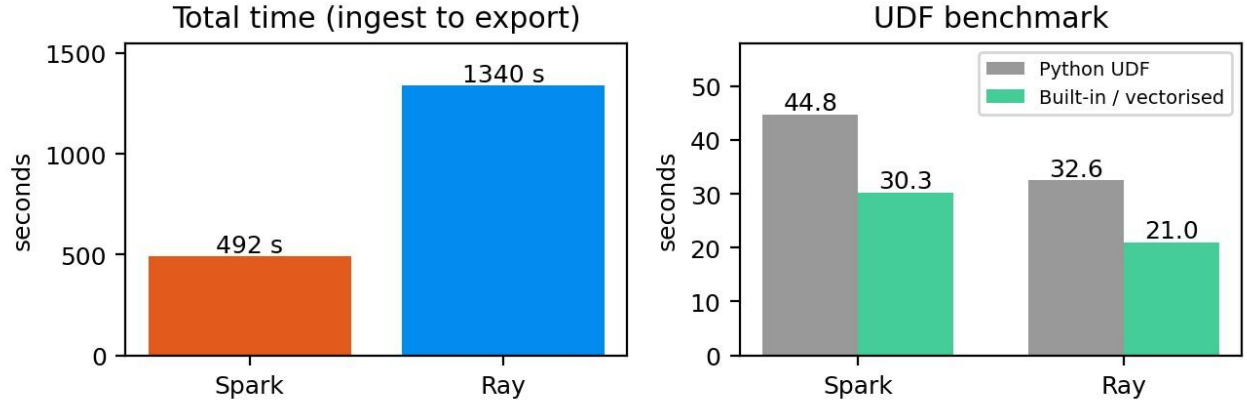


Figure 2: Left: total time. Right: time for the UDF step compared with a built-in/vectorised version (see Section 5).

4.2 Resource use

I logged docker stats every few seconds for all containers and used only the samples taken during the pipeline run. Memory numbers are reliable., I used CPU for comparing the two frameworks, not as an exact value.

Metric (during pipeline)	Spark	Ray
Peak memory, whole cluster (GiB)	8.04	8.65
Average memory, whole cluster (GiB)	5.72	7.59
Peak memory per container (GiB)	master 0.81, w1 3.50, w2 3.72	head 3.00, w1 2.93, w2 2.82
Median CPU per worker (% of 1 core, noisy)	about 1250 and 1410	about 49 and 52

Table 4: Resource use measured with docker stats.

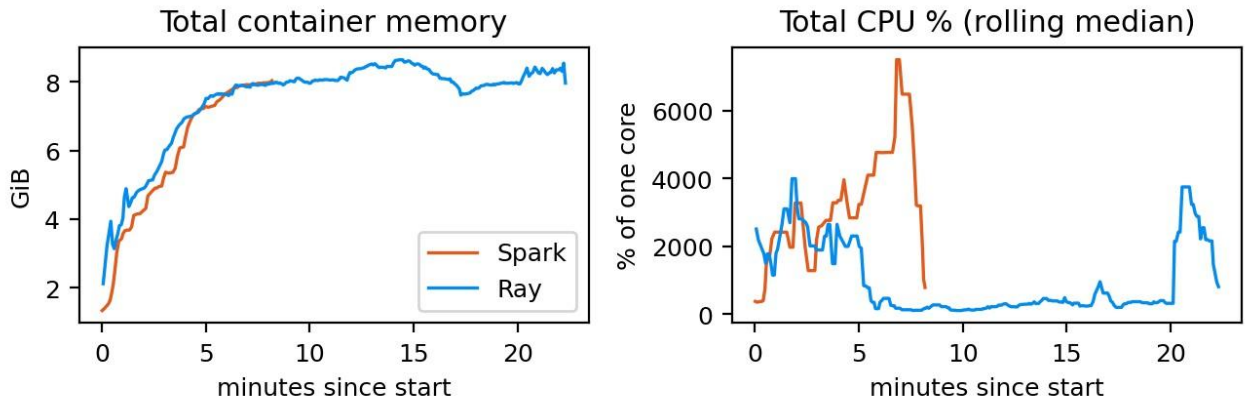


Figure 3: Memory and CPU over time (CPU line smoothed with a rolling median).

What I see in Figure 3: Spark keeps its CPUs busy for the whole 8 minutes. Ray is busy for about 5 minutes, then its CPU is almost idle from minute 6 to minute 20 while memory stays high, and it becomes busy again only at the end. The Ray head container also reached its full 3 GiB limit, which is the same time the Ray dashboard showed the head at 98% memory. During this middle phase Ray was doing the duplicate removal (a shuffle) and spilling data to disk, so it was waiting on disk and not on CPU. The raylet log also reported 14 workers killed for running out of memory (13 + 1), and Ray had to retry those tasks. This is the main reason why Ray was slower on my small VM.

5 UDF Deep-Dive

The custom function avg speed mph(distance, hours) is the same in both scripts. To measure its cost alone, I ran it on the already cleaned output and compared it with a built-in/vectorised calculation of the same value.

Framework	Python UDF (s)	Built-in (s)	Overhead (s)	Overhead (%)
Spark	44.77	30.26	14.51	48
Ray	32.56	21.01	11.55	55

Table 5: UDF overhead (UDF time minus built-in time).

In Spark, data lives in the JVM. For a Python UDF, rows must be serialised, sent to a separate Python worker process, computed, and sent back. In Ray the data already sits in pandas batches inside Python worker processes, so the function runs directly with no JVM-to-Python crossing. This matches the numbers: Ray's UDF step took 12.2 s less in total and had about 20% less overhead in seconds (11.55 s vs. 14.51 s).

But I should be honest: the difference is small. As a percentage, the overhead is similar (48% for Spark and 55% for Ray). Each number is a single run, and both tests re-read the data from disk, so I/O noise is included. In an earlier Ray attempt, the "native" timing was five times slower than normal and gave a negative overhead, which showed me how noisy this measurement can be. So I believe Ray has a small Python-native advantage, but my data cannot prove a large one. The bigger cost was not the UDF itself; the UDF is only 33–45 s in a 492–1340 s pipeline.

6 Conclusion

Winner: Apache Spark for this task on this hardware. It finished in 492 s against 1340 s for Ray, used less memory on average, did not need any task retries, and gave exactly the same output. Spark has a built-in, well-tested shuffle and dropDuplicates, which is exactly what this cleaning job needs. Ray's Python-native UDF was a little cheaper, but this gain was small compared to the shuffle cost.

BI-first project: I would choose Spark. The work is mostly SQL-style cleaning, joins and aggregations on tables, and Spark handles these quickly and with less tuning. **AI-first project:** I would choose Ray, because it runs plain Python and fits the rest of an ML workflow (feature code, GPUs, training and serving in one system), as long as the cluster has enough memory. A good combination in practice is Spark for the heavy table cleaning and Ray for ML training.